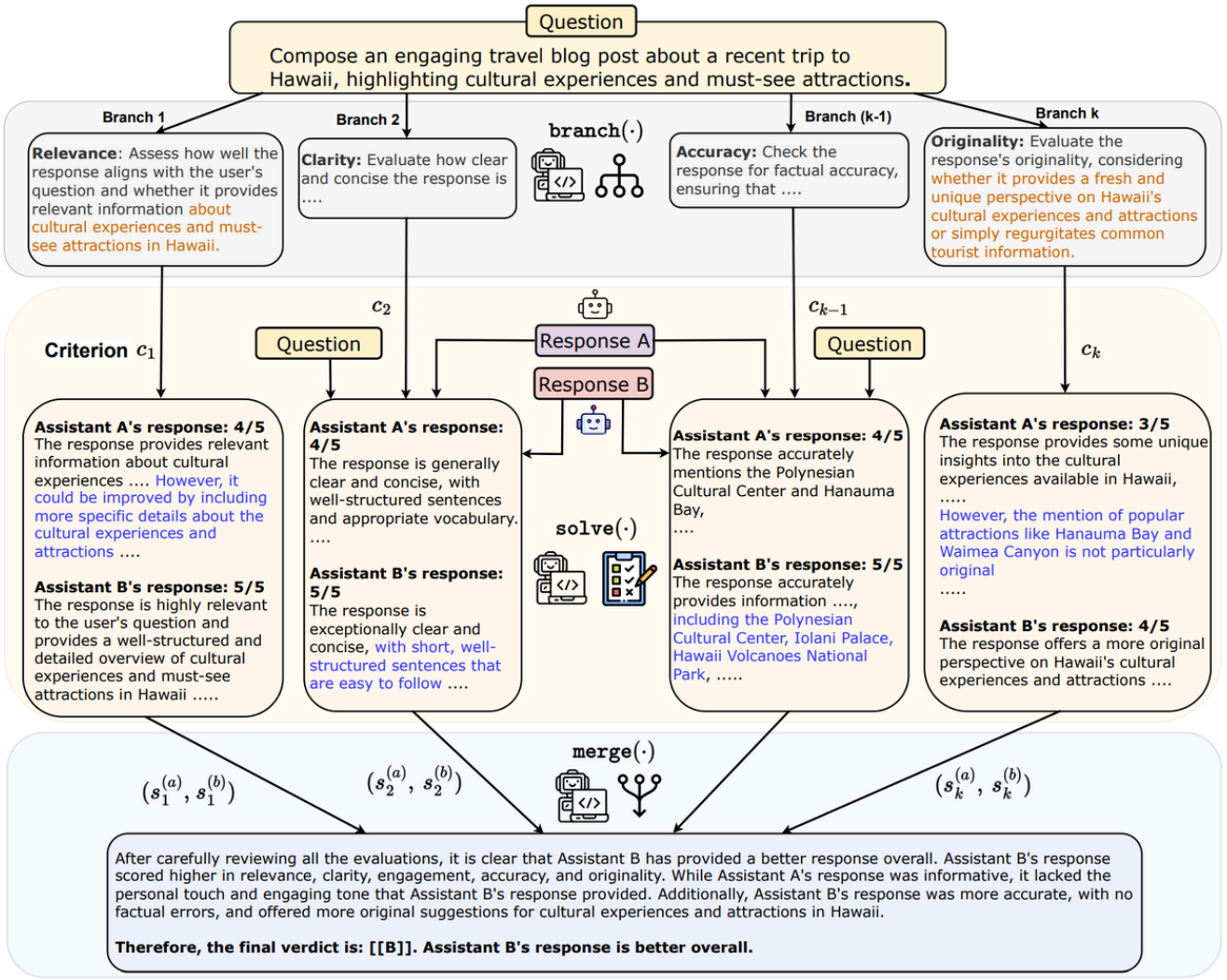
Branch-Solve-Merge Improves Large Language Model Evaluation and Generation

> #Inference



- What problems does this framework solve?
  - When LLMs try to solve complex tasks that need multiple-step planning and fulfill multi-dimensional constraints, they behave poorly as it is not consecutive or self-consistent and lacks the ability to break-down and plan *(during the time this paper released)*.
- What does this framework do?
  - This framework takes advantage of the idea of divide-and-conquer.
  - LLMs will play three roles in this framework: **Branching**, **Solving**, and **Merging**.
  - Branching: Based on the branching prompt, the model splits the task into multiple sub-tasks. The brancher will determine how many sub-tasks to generate.
  - Solving: Each sub-task will be handled by an LLM with the solving prompt in parallel.
  - Merging: Solutions to sub-tasks will be collected and compiled by the merger to generate a final answer.
  - Examples:

        - Evaluator:
            - Brancher: Generate the criteria plan of evaluation with the corresponding guides.
            - Solver: Independently run evaluation with the given perspective and the given guide.
            - Merger: Collect the result from the solver. To avoid positional bias, the system will run twice. When the 2-rounds have the same conclusions, then the evaluation is effective; otherwise the merger will return "tied".
        - Write a story with given words
            - Brancher: Split the word table into several groups; come up with a topic.
            - Solver: Independently write a story using the given word group and the topic.
            - Merger: Merge each part of the story together to ensure the fluency of the story.
- Experiment Results
    - In LLM response evaluation tasks: BSM significantly enhances “accuracy,” boosting consistency between open-source models (such as LLaMA-2-70B-chat) and human evaluators by up to 26%. It even matches or surpasses GPT-4 in numerous domains like mathematics and programming. Simultaneously, it substantially improves “consistency.” Its unique decomposition and position-swapping mechanisms reduce positional bias and length bias by up to 50%, even making GPT-4 more objective when evaluating its own responses.
    - For constrained text generation tasks: BSM-generated stories demonstrate far superior coherence compared to baseline models. When compared against zero-shot baselines, 93% of BSM stories were judged by GPT-4 to be better written. Furthermore, through divide-and-conquer, it increased the proportion of successfully incorporating all constraint words by 12%.
- My Thought
    - This structure is good for increasing the efficiency of tool-call tasks.
    - Call tools in parallel (asynchronizely) decrease the time and latency of the process. However, not all tool-calls can be run in parallel, so an **assembler** should be added after the brancher, which will group the minimum amount of sequential tool-calls into the same sub-task.
    - This can ensure both correctness and parallelism.
- Work Cited
    - Paper
    - NotebookLM
    - Chat History